

Progetto Basi Di Dati 2024/2025

Alessandro Dal Ceredo (mat. 892355) - Gabriele D'Amato (mat. 890654)

BD AIRLINE è un sistema web per la gestione di prenotazioni di voli aerei, sviluppato con Flask e PostgreSQL. L'applicazione supporta due tipologie di utenti, compagnie aeree e passeggeri con funzionalità dedicate.

L'applicazione web gestisce tutte le funzioni necessarie per la ricerca dei voli e prenotazioni per i passeggeri, e la gestione e aggiunta di voli per le compagnie aeree.

Funzionalità principali

Gestione utenti e autenticazione

- La registrazione dei due utenti è differenziata
- Alcune pagine hanno bisogno del login di un specifico tipo di utente per essere visualizzate
- Le password vengono hashate per una maggiore sicurezza (Abbiamo usato la libreria Werkzeug per hashare password e controllare se due hash sono uguali)

Gestione dei voli per le compagnie aeree

- Aggiunta e gestione aerei specificando il numero di posti per classe (economy, business e first)
- Aggiunta e gestione tratte specificando aeroporto di partenza e d'arrivo
- Aggiunta e gestione voli specificando aereo, tratta, data e orario di partenza, data e orario d'arrivo e prezzi dei posti per ogni classe
- Visualizzazione semplice dalla dashboard di informazioni utili come i ricavi totali, voli attivi, aerei in flotta e tratte servite
- Visualizzazione dei voli con filtri per tratta, aereo, data di partenza e posti disponibili

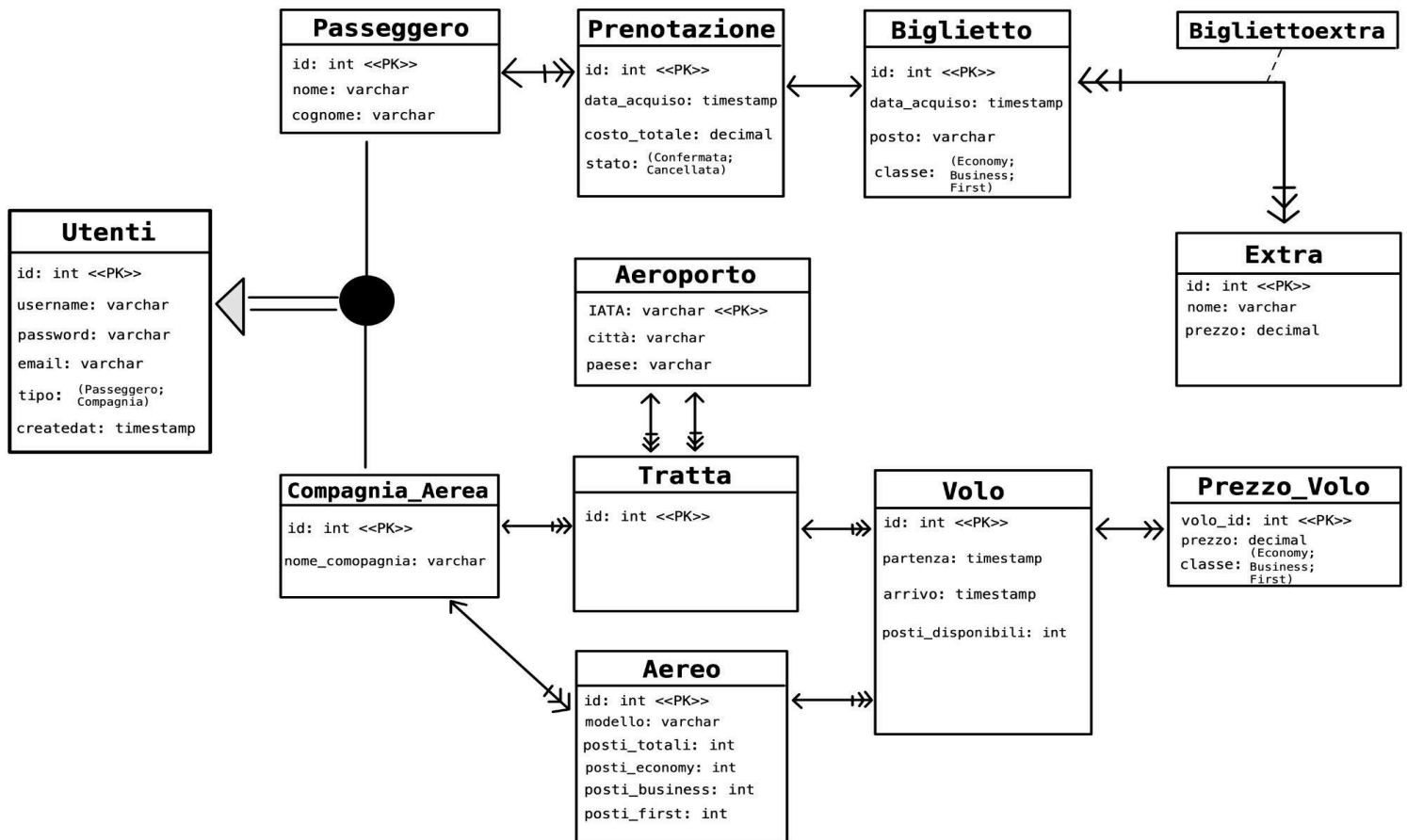
Gestione delle prenotazioni per i passeggeri

- Prenotazione di un aereo specificando classe, posto ed extra (bagaglio aggiuntivo, ...)
- Possibilità di cancellare una prenotazione solo se prima delle 24 ore dalla partenza
- Visualizzazione dei dettagli delle prenotazioni effettuate

Funzionalità generali

- Ricerca voli da entrambi i tipi di utenti e da anonimo specificando aeroporto di partenza, aeroporto d'arrivo, data di partenza e se diretti o con scali
- Visualizzazione dei posti disponibili nei voli
- Visualizzazione delle tratte più popolari nella homepage
- Ordinamento dei voli cercati per prezzo, orario di partenza e durata del volo

Progettazione Concettuale del DB



Abbiamo progettato il database con una gerarchia di specializzazione, con la tabella Utente come entità principale che contiene i dati comuni di autenticazione, mentre le tabelle CompagniaAerea e Passeggero estendono questa base con attributi specifici.

La struttura segue che gli aeroporti sono entità indipendenti, le tratte rappresentano collegamenti tra aeroporti operati da una compagnia specifica e i voli sono le istanze concrete di una tratta. Questo ci consente di riutilizzare le tratte per voli multipli e facilita la gestione degli orari e delle tariffe.

Ogni aereo appartiene a una compagnia e ha i posti suddivisi per classe, permettendo prezzi differenti.

La tabella PrezzoVolo ha la chiave composita (volo_id, classe) per permettere tariffe diverse per ogni combinazione volo-classe.

Il sistema di prenotazione separa la Prenotazione che rappresenta la transazione del passeggero, mentre ogni Biglietto è un documento di viaggio specifico.

Modello Logico Relazionale del DB

Utente (id: int, username: string, password: string, email: string, tipo: tipo_utente, createdat: timestamp)
PK(id)

CompagniaAerea (id*: int, nome_compagnia: string)
PK(id)
id FK(Utente)

Passeggero (id*: int, nome: string, cognome: string)
PK(id)
id FK(Utente)

Aeroporto (codice_IATA: string, città: string, paese: string)
PK(codice)

Tratta (id: int, aeroporto_partenza: string, aeroporto_arrivo: string, compagnia_id*: int)
PK(id)
aeroporto_partenza FK(Aeroporto), aeroporto_arrivo FK(Aeroporto), compagnia_id
FK(CompagniaAerea)

Aereo (id: int, modello: string, posti_totali: int, posti_economy: int, posti_business: int, posti_first: int, compagnia_id*: int)
PK(id)
compagnia_id FK(CompagniaAerea)

Volo (id: int, tratta_id: int, aereo_id: int, partenza: timestamp, arrivo: timestamp, posti_disponibili: int)
PK(id)
tratta_id FK(Tratta), aereo_id FK(Aereo)

PrezzoVolo (volo_id*: int, classe: classe_volo, prezzo: numeric)
PK(volo_id, classe)
volo_id FK(Volo)

Prenotazione (id: int, passeggero_id*: int, data_acquisto: timestamp, costo_totale: numeric, stato: stato_prenotazione)
PK(id)
passeggero_id FK(Passeggero)

Biglietto (id: int, prenotazione_id: int, volo_id: int, classe: classe_volo, posto: string)
PK(id)
prenotazione_id FK(Prenotazione), volo_id FK(Volo)
Extra (id: int, nome: string, prezzo: numeric)
PK(id)

BigliettoExtra (biglietto_id: int, extra_id: int)
PK(biglietto_id, extra_id)
biglietto_id FK(Biglietto), extra_id FK(Extra)

Query principali

Le query le scriviamo sia in python che abbiamo usato grazie a SQLAlchemy ORM e il suo corrispettivo in PostgreSQL

- **Ricerca voli diretti con filtri e ordinamento**

```
voli_query = db.session.query(Volo).join(Tratta).filter(
    Tratta.aeroporto_partenza == origine.upper(),
    Tratta.aeroporto_arrivo == destinazione.upper(),
    db.func.date(Volo.partenza) == db.func.cast(data_partenza, db.Date),
    Volo.posti_disponibili > 0
).order_by(Volo.partenza)
```

```
SELECT v.id, v.partenza, v.arrivo, v.posti_disponibili,
       t.aeroporto_partenza, t.aeroporto_arrivo,
       a.modello AS aereo_modello,
       c.nome_compagnia,
       pv.classe, pv.prezzo
FROM volo v
JOIN tratta t ON v.tratta_id = t.id
JOIN aereo a ON v.aereo_id = a.id
JOIN compagnia_aerea c ON t.compagnia_id = c.id
LEFT JOIN prezzo_volo pv ON v.id = pv.volo_id
WHERE t.aeroporto_partenza = %s
      AND t.aeroporto_arrivo = %s
      AND DATE(v.partenza) = %s
      AND v.posti_disponibili > 0
ORDER BY v.partenza;
```

- **Ricavi totali per compagnia**

```
ricavi = db.session.query(
    func.sum(Prenotazione.costo_totale)).join(
    Biglietto, Prenotazione.id == Biglietto.prenotazione_id).join(
    Volo, Biglietto.volo_id == Volo.id).join(
    Tratta, Volo.tratta_id == Tratta.id).filter(
    Tratta.compagnia_id == compagnia_id,
    Prenotazione.stato == 'confermata'
).scalar()
```

```
SELECT COALESCE(SUM(pr.costo_totale), 0) AS ricavi_totali
FROM prenotazione pr
JOIN biglietto b ON pr.id = b.prenotazione_id
JOIN volo v ON b.volo_id = v.id
JOIN tratta t ON v.tratta_id = t.id
WHERE t.compagnia_id = %s
AND pr.stato = 'confermata';
```

- Le 3 destinazioni più richieste

```
destinazioni_popolari = db.session.query(
    Tratta.aeroporto_arrivo,
    func.count(Biglietto.id).label('num_prenotazioni')).join(
    Volo, Tratta.id == Volo.tratta_id).join(
    Biglietto, Volo.id == Biglietto.volo_id).join(
    Prenotazione, Biglietto.prenotazione_id == Prenotazione.id).filter(
    Prenotazione.stato == 'confermata' # Solo prenotazioni confermate).group_by(
    Tratta.aeroporto_arrivo).order_by(
    func.count(Biglietto.id).desc()
).limit(3).all()
```

```
SELECT t.aeroporto_arrivo,
       a.citta, a.paese,
```

```

COUNT(b.id) AS num_prenotazioni
FROM tratta t
JOIN aeroporto a ON t.aeroporto_arrivo = a.codice
JOIN volo v ON t.id = v.tratta_id
JOIN biglietto b ON v.id = b.volo_id
JOIN prenotazione pr ON b.prenotazione_id = pr.id
WHERE pr.stato = 'confermata'
GROUP BY t.aeroporto_arrivo, a.citta, a.paese
ORDER BY num_prenotazioni DESC
LIMIT 3;

```

- Posti occupati di un volo

```

posti_occupati = (
    db.session.query(Biglietto.posto).join(
        Prenotazione).filter(
            Biglietto.volo_id == volo.id,
            Prenotazione.stato != 'cancellata'
        ).all()
)

```

```

SELECT b.posto
FROM biglietto AS b
JOIN prenotazione AS p
    ON p.id = b.prenotazione_id
WHERE b.volo_id = %s
    AND p.stato <> 'cancellata';

```

Principali scelte progettuali

Per il progetto ci serviva che il database avesse politiche di integrità molto “invasive” perchè non si può sapere cosa possano inserire potenziali utenti (ad esempio) nella registrazione dell’utente, ed impedire errori da parte della compagnia nell’inserimento di tratte, aerei o voli (esempio: tratte con aeroporto d’arrivo e destinazione uguali).

Sono stati usati vincoli di integrità di ennuola come **NOT NULL** e **DEFAULT** per molti attributi delle tabelle, questo è necessario (per esempio) per evitare di avere utenti nel database con valori NULL per il nome, email, password...

Vincoli di univocità e chiavi primarie, ogni tabella ha una chiave primaria e spesso con **SERIAL** per averli distinti in ordine di inserimento e alcuni attributi hanno **UNIQUE** tipo email e username degli utenti.

Vincoli di propagazione come **ON DELETE CASCADE** per evitare che entità dipendenti come Passeggero -> Prenotazione -> Biglietto -> BigliettoExtra possano essere eliminate correttamente e **ON DELETE RESTRICT** per evitare di cancellare una riga dalla tabella genitore che ha riferimenti nella tabella figlia tipo Compagnia_Aerea -> Aereo.

Check e Trigger per controlli extra per far funzionare il database nel modo corretto, ad esempio un check per controllare in una tratta l'aeroporto di partenza sia diversa da quella di destinazione:

```
CONSTRAINT tratta_check CHECK (aeroporto_partenza <> aeroporto_arrivo)
```

o che un aereo abbia almeno un posto e che sia uguale alla somma dei posti di ogni classe:

```
CONSTRAINT aereo_posti_totali_check CHECK (posti_totali > 0),
```

```
CONSTRAINT aereo_check CHECK (posti_economy >= 0 AND posti_business >= 0 AND  
posti_first >= 0),
```

```
CONSTRAINT aereo_check1 CHECK (posti_economy + posti_business + posti_first = posti_totali)
```

Alcuni trigger sono stati implementati per controllare la coerenza sulla compagnia tra tratta e aereo quando si inserisce o modifica un volo e due trigger per decrementare o incrementare posti disponibili quando si effettua una prenotazione o se ne cancella una:

```
CREATE OR REPLACE FUNCTION fn_dec_posti_disponibili() RETURNS trigger AS $$  
BEGIN
```

```
    UPDATE volo SET posti_disponibili = posti_disponibili - 1
```

```
    WHERE id = NEW.volo_id;
```

```
    RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
DROP TRIGGER IF EXISTS trg_dec_posti_after_insert ON biglietto;
```

```
CREATE TRIGGER trg_dec_posti_after_insert
```

```
AFTER INSERT ON biglietto
```

```
FOR EACH ROW EXECUTE FUNCTION fn_dec_posti_disponibili();
```

Altri check e verifiche meno critiche sono state aggiunte nell'applicazione scritta in python per essere più facilmente modificabili.

Le politiche di autorizzazione vengono gestite nell'applicazione scritta in python che controlla prima di accedere alle pagine se l'utente connesso alla sessione di flask sia di tipo passeggero o compagnia aerea.

Abbiamo creato degli indici per le ricerche più frequenti. Un indice per voli di una certa tratta in una certa data:

```
CREATE INDEX idx_volo_tratta_partenza
```

```
ON volo (tratta_id, partenza);
```

Un indice per velocizzare la query nella funzione fn_validate_ticket_capacity():

```
CREATE INDEX idx_biglietto_volo_classe  
ON biglietto (volo_id, classe);
```

Un indice per visualizzare la cronologia acquisti di un passeggero:

```
CREATE INDEX idx_prenotazione_passeggero  
ON prenotazione (passeggero_id, data_acquisto DESC);
```

e altri...

Ulteriori informazioni

DBMS: PostgreSQL

Backend: Python, Flask, SQLAlchemy

Frontend: HTML, CSS, Javascript

Librerie usate

Flask

Flask-SQLAlchemy

Flask-Login

Werkzeug (libreria usata per hashare password e controllare se due hash sono uguali)

SQLAlchemy

Altre implementazioni

Una classe config per facilitare il lavoro di gruppo sul progetto senza dover scrivere direttamente in nel file app.py l'host, porta, utente e password del database postgresQL

Sono state create pagine di errore 404 e 500

```
@app.errorhandler(404)  
def not_found_error(error):  
    return render_template('404.html'), 404  
  
@app.errorhandler(500)  
def internal_error(error):  
    db.session.rollback()  
    return render_template('500.html'), 500
```

Il progetto è stato completato insieme dato che vivendo nella stessa città c'è stato possibile trovarci e lavorare sul progetto in contemporanea. (C'è stata una maggior cura per il Front-End e User-Experience da parte di Alessandro Dal Ceredo, mentre per il Back-End da parte di Gabriele D'Amato)